

Abstract:

This document explains how our content server network was built and how it works. Each computer runs the same program, `content_server.py`, and together they form a network that can figure out its own shape and find the shortest paths between nodes. To know if a neighbor is still online, each node sends a short "alive" message to all its neighbors every 0.5 seconds. If a neighbor goes more than 5 seconds without sending one, it is marked as inactive and removed from the neighbor list.

Once a node has the full network map, it uses Dijkstra's algorithm to find the shortest distance to every other node. The algorithm keeps a set of unvisited nodes and repeatedly picks the closest one, updating distances as it goes. The algorithm breaks when there are no unvisited nodes left to visit.

KeepAlive Message Implementation

```
msg = ("Alive:" + self.uuid + ":" + str(time.time()) + ":" + str(self.backendPort) + ":" + str(info['metric']))
```

My implementation of the keepalive message is here. It contains the header, "Alive" which allows any nodes to know that this is a "Alive" type of message, it has the uuid of the sender, the time it was sent, and the port, and metric in case the listener does not have that uuid in their system. This creates a process where if

- 1) The receiver already knows the sender, it updates the lastAlive time to ensure that the connection is still secure
- 2) If the receiver does not know the sender, the uuid, port, and metric feature allows the receiver to add the node to its neighbor list

LinkState Message Implementation

```
MSG = "Link State Packet:" + self.uuid + ":" + self.name + ":" + str(self.ownSeqNumbers) + ":" + neighbor_string
```

Like Keep Alive, the message starts off with "Link State Packet" where it allows any receivers to know that instead of an "Alive" message, it is a Link State message. This creates a process where if:

1. The sequence number is lower than or equal to the last one seen from that uuid, the message is discarded since it is old or duplicate information.
2. The sequence number is higher than the last one seen, the receiver updates `self.map[name]` with the new neighbor list, saves the new sequence number, and forwards

the same message to all of its own neighbors so the information continues to spread across the network.

As each node sends this to every neighbor every 0.5 seconds, the sequence number helps with distribution as it acts like a “version control” where only the latest information gets distributed throughout the network.